

SAMSUNG PRISM · GENERATIVE AI HACKATHON · 3RD
EDITION

Streaming Live RAG — Benchmark Report

Team Nexora

SRM Institute of Science and Technology, Chennai

October 2026

SL-RAG Benchmark Report

Setup

Item	Value
Configuration	<code>config.yaml</code> , <code>frozen: true</code> , <code>cfg_hash e3745a51d783adef</code> (stamped in <code>out/summary.json</code> and <code>out/final_experiments.json</code>)
Split	test: 39 scenarios + 8 late-detail sessions = 47 sessions, 55 turns (seed 13, temperature 0)
Calibration	controller thresholds and the uncertainty thresholds tuned on the tune split only (36 + 7 sessions), then frozen
Corpus	24 synthetic Nexora documents (93 sections) plus the 8-section sample corpus: 101 indexed chunks
Retrieval	BM25 + dense (<code>sentence-transformers/all-MiniLM-L6-v2</code> , CPU), fused with RRF (k = 60; weights BM25 0.4, dense 0.6)
LLM	<code>heuristic</code> backend (deterministic stand-in drafter, no model server)
Timing	virtual replay clock (retrieval 120 ms, draft 350 ms, verify 15 ms, chunk interval 200 ms): relative, not hardware latency
Cost	token estimates × configured prices (\$0.00015 / \$0.0006 per 1k in / out): notional
Baselines	B1 : hybrid retrieval and verifier, but batch (retrieves only at utterance end), no planner, restarts on a late detail. B0 : as B1 but dense-only and without the verifier.

Reproduce with:

- `python scripts/fetch_models.py`, which downloads the retrieval model once;
- `python -m slrag.cli replay eval/scenarios --split test --out out/ours_test.jsonl`, which writes `out/summary.json` and `out/report.md`;
- `python scripts/run_experiments.py`, which writes `out/final_experiments.json`.

1. Metrics: Ours vs baselines (test split)

#	Metric	Ours	B1	B0
1	recall@5	0.989	0.910	0.897
2	recall@10	1.000	0.962	0.923
3	groundedness (verifier score)	1.000	1.000	n/a (no verifier)
4	hallucinated-ID rate	0.000	0.000	0.000
5	TTFT p50 (virtual ms)	388	485	485
6	TTFT p95 (virtual ms)	490	590	590
7	cost per turn (notional \$)	0.000424	0.000179	0.000174
8	early-retrieval rate	0.950	0.000	0.000
9	false-trigger rate	0.000	0.000	0.000
10	sub-intent recall	0.951	0.590	0.574
11	over-fragmentation	0.077	0.000	0.000
12	suppression precision / recall	0.727 / 1.000	0.727 / 1.000	0.667 / 1.000

#	Metric	Ours	B1	B0
13	uncertainty precision / recall	0.727 / 1.000	0.727 / 1.000	0.667 / 1.000
14	claim preservation on late details	1.000	n/a (restarts)	n/a (restarts)
15	refinement savings, turn-2 retrieval calls (A4)	56.3% fewer (7 vs 16)	baseline	n/a
16	trace coverage	1.000	0.952	0.821

Where Ours is worse than the baselines:

- It costs 2.4× as much per turn as B1, because of speculation and pre-drafting.
- It over-fragments 7.7% of single intents (failure 2).

2. Gates G1–G6 (test split, `out/summary.json`)

Gate	Condition	Measured	Status
G1 recall improvement	recall@10 Ours ≥ B1	1.000 ≥ 0.962	✓ PASS
G2 early retrieval	early_retrieval_rate ≥ 0.80	0.950	✓ PASS
G3 groundedness preservation	groundedness Ours ≥ B1	1.000 ≥ 1.000	✓ PASS
G4 grounding	groundedness ≥ 0.85 and hallucinated-ID rate = 0	1.000, 0.000	✓ PASS
G5 late-detail refinement	preservation = 1.0, restarts = 0, lineage complete	1.000, 0, 1.000	✓ PASS
G6 trace coverage	coverage = 1.0	1.000	✓ PASS

3. Changes since the previous freeze (`cfg_hash 90e68e6053d494aa`)

Change	Calibrated on	Effect on the test split
Dense retrieval: hashed n-gram projection → <code>all-MiniLM-L6-v2</code>	tune: controller grid re-run; the choice (0.86, 0.82) is unchanged, and tune recall@10 rose from 0.919 to 1.000	recall@10 0.926 → 1.000 , recall@5 0.872 → 0.989, sub-intent recall 0.918 → 0.951
Uncertainty: <code>sufficiency.uncertain_high</code> 0.70 → 0.50 (empty band); <code>dense_top1</code> stays 0.55	tune: <code>scripts/calibrate_uncertainty.py</code> , 50-point grid, uncertainty F1 1.000	uncertainty precision 0.320 → 0.727 (recall 1.000)
Refinement prompts: JSON schemas removed from the prompt text, fields named inline (the schema still validates every reply and is Ollama's <code>format</code>)	tune: 17.5% fewer refinement tokens	refinement tokens per turn 3,082 → 2,526 (–18.1%) in the same run; vs the previous report's 2,761: –8.5% (the planner now also receives each claim's cites)
Verifier semantic check: compare against the best-matching sentence of the cited text as well as the whole chunk	tune: G3 failed (Ours 0.9615 vs B1 0.9643); all 25 rejected claims were verbatim chunk sentences	groundedness 0.966 → 1.000 (B1 also 1.000); fabricated claims still score 0.55–0.59 against the 0.65 threshold
Delta planner input: each claim's cited chunk ids	tune; this restored a regression in the refinement end-to-end tests	relation accuracy 1.000, affected accuracy 0.875

Disclosure. The test split was measured twice under the new retriever:

1. **First run:** G3 failed (Ours 0.966 vs B1 0.977).
2. **Root cause:** the semantic-check defect above. It was diagnosed and fixed on the tune split, which showed the same miss.
3. **Second run:** the config was refrozen and the whole final experiment re-run. No threshold was chosen by looking at test results.

The first run is preserved in `out/summary_minilm_before_verifier_fix.json`, and the previous freeze in `out/summary_hash_embedder.json`.

4. Ablations A1–A5 (`out/final_experiments.json`)

Each arm runs the full "ours" pipeline with exactly one setting changed.

Before / after the retriever swap

The ablations were run on the previous freeze (hashed n-gram dense retriever, `cfg_hash 90e68e6053d494aa`) and again on the current one (MiniLM, `cfg_hash e3745a51d783adef`):

Ablation	Metric	Before	After
A1	recall@10: hybrid / dense-only / BM25-only	0.926 / 0.809 / 0.989	1.000 / 0.968 / 0.989
A2	early retrieval: cascade vs eager	0.950 vs 0.888	0.950 vs 0.888 (unchanged)
A3	TTFT p50, ms: off / retrieval_only / full	490 / 385 / 388	490 / 385 / 388 (unchanged)
A4	turn-2 tokens: refinement vs restart (ratio)	22,085 vs 5,670 (3.9×)	20,204 vs 5,715 (3.5×)
A4	delta recall@5	0.875	1.000
A5	claims supported on re-check, verifier off	137 of 139 (98.6%)	154 of 154 (100%)

A1: Retrieval (hybrid vs dense-only vs BM25-only)

Arm	recall@5	recall@10	sub-intent recall	groundedness	uncertainty P
hybrid (shipped)	0.989	1.000	0.951	1.000	0.727
dense only (MiniLM)	0.947	0.968	0.951	1.000	0.727
BM25 only	0.936	0.989	0.984	1.000	0.889

Finding. With a real embedding model, hybrid fusion beats both of its parts on recall@5 and recall@10. With the previous hashed n-gram retriever, BM25-only had won (0.989 vs 0.926).

BM25-only still has the best sub-intent recall and uncertainty precision. Exact-term matching helps sub-intent recall on short sub-queries. On uncertainty, BM25 scores give the coverage gate fewer near-miss chunks (see failure 1).

A2: Controller (eager vs rules_only vs cascade vs llm_only)

Arm	early retrieval	false trigger	wasted speculation	cache hit	TTFT p50 / p95	cost / turn	recall@10
eager	0.888	0.000	0.313	0.392	385 / 490	0.000316	1.000
rules_only	0.950	0.000	0.223	0.458	388 / 490	0.000424	1.000
cascade (shipped)	0.950	0.000	0.223	0.458	388 / 490	0.000424	1.000
llm_only	0.950	0.000	0.223	0.458	388 / 490	0.000424	1.000

Finding. cascade, rules_only and llm_only are identical, and all 290 controller decisions in the test run came from tier T1:

- T0 and T2 only engage once a session already has an answer.
- In `llm_only` mode, the heuristic T2 classifier answers RETRIEVE, which hands the decision to T1.

The cascade's T0/T2 tiers are therefore unmeasured by this harness. Eager dispatch wastes more speculation (0.313 vs 0.223).

A3: Speculation (off vs retrieval_only vs full)

Arm	TTFT p50 / p95	cost / turn	groundedness	early retrieval	pre-draft tokens wasted
off	490 / 490	0.000316	1.000	0.000	n/a
retrieval_only	385 / 490	0.000315	1.000	0.950	n/a
full (shipped)	388 / 490	0.000424	1.000	0.950	18,944 (61.5%)

Finding. Early retrieval is where the latency win comes from: 105 ms off the p50. Pre-drafting does not improve TTFT on this split:

- Of 29 pre-drafts, 13 stood, 0 were refined and 16 were redone.
- Only 34.5% were ready before the utterance ended.
- It adds 35% to cost per turn.

`retrieval_only` is the better operating point on these numbers. The frozen config keeps `full`.

A4: Late-detail refinement (delta) vs restart

	turn-2 retrieval calls	turn-2 tokens
refinement (ours)	7	20,204
restart (B1)	16	5,715

- Over 8 sessions, refinement makes 56.3% fewer retrieval calls and preserves every unaffected claim.
- It still uses **3.5× more tokens** (was 3.9×; failure 3).
- Delta-planner relation accuracy is 1.000, affected-sub-intent accuracy 0.875, and delta recall@5 1.000 (was 0.875).

A5: Fail-closed verifier (on vs off)

Arm	groundedness	emitted claims	supported on post-hoc re-check	turns with an answer
on (shipped)	1.000	491	491 (100%)	39
off	n/a	154	154 (100%)	19

Finding. With the extractive heuristic drafter, every claim is a sentence copied from retrieved evidence, so there is nothing for the verifier to catch: both arms are 100% supported.

The verifier's value shows only with a generative drafter, such as the Ollama backend, which was not measured.

Caveat. This is not a clean ablation: with `enable_verifier: false`, unverified claims are recorded but only 19 of the 39 answered turns stream an answer.

5. Failure analysis (real failures from the frozen test run)

Failure 1: The coverage gate suppresses answerable paraphrased questions (presentation_06 , presentation_07 , compound_08)

- **Expected.** Each of these should be answered from its gold chunk:
 - presentation_06: "Summarize the Salesforce integration setup steps.";
 - presentation_07: "Summarize the Nexora company history since its founding.";
 - compound_08 sub_1: "What new features shipped in the v2.4 release highlights?"
- **Actual.** Retrieval found the gold chunk in the top 2 every time, but the sufficiency gate suppressed all three as insufficient evidence. These are the only 3 false flags, which gives uncertainty and suppression precision of $8 / 11 = 0.727$.
- **Root cause.** This is the sufficiency gate (slrag/pipeline/sufficiency.py), whose coverage threshold is 0.50.
 - Coverage counts the share of query content words that appear in the evidence. Instruction words ("summarize", "steps", "new", "shipped") are counted but never occur in the chunk, so these queries fall under 0.50.
 - In compound_08 , the sub-query also picked up a stray token from the next clause ("...highlights? 429"), a query-builder defect.
 - The dense scores alone separate these cases cleanly.
 - The tune split could not show this: its calibration reached F1 1.000 with no such cases.

Coverage and dense score of the three false flags, against the truly unanswerable sub-intents:

Sub-intent	Coverage	Dense top-1
compound_08 sub_1	0.375	0.731
presentation_06	0.400	0.696
presentation_07	0.429	0.804
the 8 unanswerable ones	0.00–0.29	0.558–0.597

- **Mitigation.**
 - Accept a sub-intent whose dense top-1 is high even when coverage is low.
 - Drop presentation verbs from the coverage count.

Either needs tune scenarios with paraphrased requests before it can be calibrated. It was not changed here, because the config is frozen and the test split must not drive tuning.

Failure 2: Over-fragmentation into a pronoun-only sub-query (compound_26)

- **Expected.** Two sub-intents:
 1. "How many employees does Nexora have and where are its offices?" (one chunk, nx-company\$team-offices);
 2. the Salesforce retry question.
- **Actual.** The planner split the first intent at "and" into "How many employees does Nexora have" and the pronoun-only "where are its offices?", giving 3 sub-intents. This is the over-fragmentation case (rate 0.077).
 - With MiniLM, retrieval now recovers: all three sub-queries are answered, and the gold chunks are found.
 - The cost is padding: the answer cites 11 chunks, including unrelated ones such as nx-company\$funding , nx-privacy-gdpr\$data-residency and nx-security\$access-control .

- **Root cause.** This is the planner's coordinator split (`s1rag/plan/splitter.py`). It splits on "and" even when the right-hand clause has only a pronoun subject, and it does not carry the antecedent ("Nexora") into that clause.
- **Mitigation.** Do not split when the right-hand clause's subject is a pronoun, or substitute the antecedent before retrieval. Left: a planner change needs a tune-split re-run.

Failure 3: Refinement still costs more tokens than restarting (A4)

- **Expected.** Refining only the affected claims should be cheaper than restarting.
- **Actual.** Turn-2 refinement used 20,204 tokens vs 5,715 for restart (3.5×), although it made 7 vs 16 retrieval calls.
- **Root cause.** This is the Phase 7 refinement path: the delta planner and the rewriter.
 - Removing the JSON schemas saved 18.1%.
 - What remains is the delta planner input: 9,358 of the 20,204 tokens, which carries every session claim with its cites.
 - The rewriter accounts for 10,536 tokens: the affected claims plus up to 4 evidence chunks per sub-intent.
 - Restart drafts once from 4 chunks.
- **Mitigation.** Give the planner sub-intent summaries instead of every claim, and cap the rewriter evidence at chunks not already cited. Left for a follow-up.

6. Compliance

`python scripts/compliance_audit.py` passes all six checks: corpus isolation, no hardcoding, no precomputation, session isolation, parsimony and prompts. It writes `out/compliance.json`.

- **Model loading.** Corpus isolation now also requires every Hugging Face model load to use `local_files_only=True`. The retrieval model is downloaded once, at setup or Docker build, and never fetched at run time.
- **Gold labels.** The no-precomputation check replays the full test split with every gold label stripped and requires identical engine decisions.

7. What this report does not claim

-
- Absolute latency on any hardware: all latency is virtual time. Loading MiniLM adds startup time, which is not modelled.
 - Real-dollar cost.
 - "Zero hallucination" in general. Groundedness is the verifier's rule-based score over an extractive drafter, and no human-labelled sample or LLM judge was run.
 - Performance with the Ollama backend: it was not measured.
 - Anything about a private held-out set.